COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT DOCUMENTATION

Team ID	SWTID-2026-3296
Team Size	4
Team Leader	Kamalesh P R
Team Member	JAYAKUMARAN V M
Team Member	Kannan P
Team Member	Naresh C

Prepared based on the supplied ComicCraft project details and formatted following the supplied sample project documentation.

1. Project Description

ComicCraft is a web-based application that uses generative AI to create personalized comic stories and illustrations from user-provided prompts. Built with FastAPI and integrated with Google Gemini models and Stable Diffusion, it automates the generation of story outlines, narration, character dialogues, and comic-style imagery.

Users provide a story prompt, main character name, setting, tone, and preferred art style. The application generates a panel-by-panel comic, displays the result in a web preview, and allows the complete comic to be downloaded as a PDF containing illustrations and narrative text.

1.1 Objectives

- Generate structured comic outlines and story narration from user prompts.
- Create comic-style illustrations for each panel using image-generation prompts.
- Provide an intuitive browser-based interface for previewing generated comics.
- Export the finished comic into a multi-page PDF for saving, printing, or sharing.

1.2 Scenarios

Scenario 1: Personalized comic generation

A user enters a prompt such as “A brave fox exploring an enchanted forest” and supplies a character name, setting, tone, and art style. Gemini Flash creates a structured outline, Gemini Pro expands it into narration and dialogue, and Stable Diffusion generates the panel illustrations.

Scenario 2: Tone and style customization

A user selects a funny tone and comic-book art style. The system regenerates the outline, story, and images according to the chosen preferences, allowing the user to iterate until the comic matches their creative vision.

Scenario 3: PDF export

After reviewing the comic preview, the user requests a download. The layout builder organizes each panel’s title, image, and text; the exporter compiles the content into a PDF using FPDF and displays an export-success page.

2. Technical Architecture

ComicCraft follows a modular three-part architecture: frontend, backend, and AI integration. The frontend collects user inputs and displays results; FastAPI manages routing and processing; Gemini and Stable Diffusion generate text and images.

2.1 Frontend

- HTML, CSS, and Jinja2 templates provide the user interface.
- index.html collects story prompt, character name, setting, tone, and art style.
- comic_preview.html displays panel titles, illustrations, scene descriptions, captions, and narration.
- export_success.html confirms that the comic PDF has been exported.

2.2 Backend

- FastAPI receives form submissions and JSON requests.
- routes.py coordinates the generation workflow and renders templates.
- Backend modules create outlines, stories, images, layouts, and PDF exports.

• Pydantic PromptRequest validates JSON-based input; HTTPException and try-except handling manage errors.

2.3 AI Integration

• Gemini Flash (models/gemini-1.5-flash): structured five-panel comic outline generation.

• Gemini Pro (models/gemini-1.5-pro): detailed narration and character dialogue.

• Stable Diffusion (runwayml/stable-diffusion-v1-5): comic-style illustrations from image prompts.

• Hugging Face Diffusers and PyTorch support image-generation workflows.

2.4 Workflow Diagram (Textual)

User input → FastAPI routes → Gemini Flash outline → Gemini Pro narration/dialogue → Stable Diffusion panel images → Layout builder → Comic preview and PDF exporter → Export success.

3. Pre-requisites

• Python programming and pip/virtual-environment setup.

• FastAPI framework and Uvicorn ASGI server.

• HTML, CSS, and Jinja2 templating.

• Google Gemini API familiarity.

• Hugging Face Diffusers, Stable Diffusion, and basic PyTorch.

• FPDF for PDF generation and Git for version control.

4. Project Workflow

Milestone 1: Model Selection and Architecture

• Activity 1.1: Review requirements for panel outlines, narration, dialogue, and illustrations.

• Activity 1.2: Select Gemini Flash, Gemini Pro, and Stable Diffusion based on creative quality, prompt adherence, speed, stability, and documentation.

• Activity 1.3: Define frontend, FastAPI backend, and AI integration responsibilities.

• Activity 1.4: Create the Python virtual environment, install dependencies, and establish the project structure.

Milestone 2: Core Functionalities Development

• Activity 2.1: Implement outline generation, story generation, image generation, layout building, and PDF export.

• Activity 2.2: Connect these modules to FastAPI routes for form and JSON inputs.

Milestone 3: routes.py Development

• Activity 3.1: Implement homepage, comic-generation, JSON API, export-success, and image-test routes.

• Integrate AI functions, process results, pass content to Jinja2 templates, and handle API or parsing errors.

Milestone 4: Frontend Development

• Activity 4.1: Build responsive input and preview pages with clear fields, buttons, and panel layouts.

• Activity 4.2: Use Jinja2 TemplateResponse to bind generated panel data, images, and export paths dynamically.

Milestone 5: Deployment

• Activity 5.1: Configure the local environment, dependencies, and API credentials.

• Activity 5.2: Launch the app locally and verify the web pages and API endpoints.

5. Core Functionalities

5.1 Generate Structured Comic Panel Outline

Function: generate_outline() **Module:** gemini_flash.py

Uses Gemini Flash to produce a structured five-panel outline. Each panel includes a panel number, title, scene description, and image-generation prompt. The result is a list of dictionaries.

5.2 Generate Story Narration and Dialogue

Function: generate_story() **Module:** gemini_pro.py

Uses Gemini Pro to expand the outline into a comic-style story with narration and character dialogue for each panel.

5.3 Generate Comic Illustrations

Function: generate_image() **Module:** image_generator.py

Uses Stable Diffusion to generate comic-style images from prompts, sanitizes filenames, saves images under static/panels, and returns each saved image path.

5.4 Build Comic Layout

Function: build_comic_layout() **Module:** layout_builder.py

Matches each generated image with its corresponding story content and creates a structured layout containing panel number, image path, and text.

5.5 Export Comic to PDF

Function: save_pdf() **Module:** exporters.py

Uses FPDF to place each panel's image and narration on separate pages. The generated PDF is saved in static/exports and its path is returned.

6. Backend Routes and Input Processing

6.1 Main Routes

• / — renders index.html.

• /generate — accepts form data and returns the comic preview after generating the comic.

• /generate-comic/json — accepts JSON validated with PromptRequest and returns layout data and PDF path.

• /export-success — displays export confirmation.

• /test-image — tests image generation directly from a custom prompt.

6.2 Input Fields

• Story prompt: the central idea of the comic.

• Character name: the main character or hero.

• Setting: the story location, such as forest, school, or city.

• Story tone: for example, dramatic, funny, or poetic.

• Art style: for example, anime, realistic, pixel art, or comic book.

6.3 Generation Sequence

The generation routes call generate_outline(), generate_story(), generate_image() for each panel, build_comic_layout(), and save_pdf(). The browser route renders comic_preview.html; the JSON route returns structured data for API clients.

7. Frontend Design and Templates

7.1 index.html

Provides a labeled form for story prompt, character name, setting, tone, and art style. It submits the data to /generate using POST.

7.2 comic_preview.html

Displays the generated comic sequentially. Each panel includes its number and title, generated illustration, italicized scene description, captions, narration, and relevant prompt reference.

7.3 export_success.html

Shows a confirmation after PDF export and can provide a control to return to the homepage and create another comic.

7.4 Dynamic Rendering

FastAPI's Jinja2Templates and TemplateResponse render pages with the current generated content. Jinja2 loops through the layout list to display panels and binds the export path to the success page.

8. Environment Setup and Deployment

8.1 Create and Activate a Virtual Environment

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate
```

8.2 Install Dependencies

```
pip install fastapi uvicorn jinja2 python-multipart google-generativeai diffusers transformers fpdf Pillow accelerate
```

8.3 Configure API Credentials

Store sensitive credentials in environment variables or a .env file and load them securely. The project details identify GEMINI_API_KEY and HF_API_KEY. Do not commit secret keys to version control.

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your-huggingface-api-key-here
```

8.4 Run the Application

```
uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000 to use the application and http://127.0.0.1:8000/docs to explore and test the FastAPI endpoints.

9. Testing and Verification

- Verify that the homepage accepts all five user inputs.

- Test outline generation and confirm that five structured panels are returned.
- Check that narration and dialogue correspond to the generated outline.
- Verify that an illustration is generated and saved for each panel.
- Check that the preview displays panel text and images in sequence.
- Verify PDF creation, file path, and export-success page.
- Test /generate-comic/json and /test-image through the interactive API documentation.
- Check error handling for API failures and malformed JSON responses.

The supplied project material describes local launch and verification steps; this document does not claim independent execution or test results.

10. Conclusion and Future Enhancements

ComicCraft demonstrates an end-to-end AI-assisted comic creation workflow. Gemini Flash and Gemini Pro generate structured outlines, narration, and dialogue, while Stable Diffusion creates comic-style illustrations. FastAPI coordinates the process, Jinja2 presents the result, and FPDF produces a downloadable comic.

The current project description focuses on comic generation without user account management. Potential future enhancements identified in the project material include user profiles, saved comic libraries, and multi-page story arcs.

— End of Document —